

Eighty years, and where we actually are

This field has collapsed publicly twice, and both times the cause was the same. You will compute, yourself, the one number that explains why the breakthrough came in 2012 and could not have come in 1989 — and then we bring the story to this month: what these systems really do, how to read a leaderboard without being fooled, and why this course is being taught in Grand-Bassam.

Unit 0 • What AI is

80 minutes

One calculation, done in class

Last session before Python

The timeline Two winters Already there Compute it What works now Leaderboards The unmeasured gap Côte d'Ivoire
Before Friday



Delivery note — this session carries what used to be two

Seventy-eight minutes of material in an eighty-minute slot, so it runs to time only if it runs to plan. Two rules if you fall behind.

Part 4 is not negotiable

— the compute calculation is the single thing this session exists to make them do, and it is the only part with no substitute in the reading.

Part 7 is the release valve

: the unmeasured-gap argument survives being cut to three sentences and a pointer to the reading, and the closing on Côte d'Ivoire must not be rushed, because it is the answer to "why am I sitting in this room".

PART 1 • 10 MINUTES

The whole history, once, quickly

Do not memorise this table. Read it for one thing: where the ideas sit relative to where the money sits.

1

Year	What happened	Why it matters to you
1943	The artificial neuron idea	McCulloch and Pitts show a simple threshold unit can compute logical functions. It is the unit you will build in session 13 — <i>thirteen years before the field has a name.</i>

Year	What happened	Why it matters to you
1950	Turing's test idea	Replaces the unanswerable "can machines think?" with something you can actually run.
1956	Dartmouth boom	The phrase "artificial intelligence" is coined at a summer workshop. Two months, about ten men, and enormous confidence.
1958	The perceptron idea	Rosenblatt builds a machine that learns its own weights from examples. The first learning-based system in this course's sense — but only one layer of weights.
1966	The ALPAC report chill	Concludes machine translation is slower, less accurate and more expensive than using human translators. US funding for it collapses.
1969	Proof of the limit chill	Minsky and Papert show a single-layer perceptron cannot learn XOR. You will reproduce this failure yourself in session 12.
1973	The Lighthill report winter	A review for the British Science Research Council finds the field has not delivered what it promised, and names combinatorial explosion as the barrier. UK funding is cut across universities.
1974–80	The first winter winter	Funding withdrawn on both sides of the Atlantic.
1975–86	Expert systems boom	MYCIN, XCON, and a commercial industry. You met these on Friday.
1986	Backpropagation idea	Rumelhart, Hinton and Williams show how to spread error across many layers. Every model in this course still trains this way. You will derive it in session 14.
1987–93	The second winter winter	The expert-system market vanishes. Companies built on it fail.
1989–98	Convolutional networks, in production proof	LeCun's networks read handwritten digits. Deployed commercially from 1993 in cheque-reading machines; by the late 1990s reading over 10% of all cheques in the United States.
1997	Deep Blue • LSTM proof	Search-based AI's high point. In the same year, Hochreiter and Schmidhuber publish the LSTM, which ran language modelling until 2017.
2012	AlexNet boom	The error rate falls off a cliff. Everything after this is downstream of it.
2017	Transformers idea	One architecture, eight authors, and the foundation of every system you will read about in the news. Session 25.
2022–26	The public era boom	ChatGPT reaches a hundred million users in two months. Then images, video, code, agents.



Read the colours, not the entries

Notice that the ideas are spread evenly across the whole table, while the booms and winters cluster. The supply of good ideas was roughly constant. What oscillated was the willingness to pay for them. Hold that thought for the rest of the session.

PART 2 • 10 MINUTES

Two winters, one cause

Two reports, eight years apart, ended the field's first golden age. Neither of them said the technology did not work.

2

Machine translation had been promised inside a decade. Russian in, English out, fully automatic. The money followed the promise. By 1966 the Automatic Language Processing Advisory Committee reported back that machine translation was slower than human translation, less accurate than human translation, and more expensive than human translation. Funding for it in the United States effectively stopped.

The deeper problem was one you have already met. In 1973 Sir James Lighthill reviewed the state of UK AI research for the British Science Research Council. His verdict was that in no area had the field produced the impact that had been promised, and his central technical objection was **combinatorial explosion** — the same wall we walked into on Friday when we computed 10^{120} for chess.

THE ARGUMENT THAT CLOSED THE LABS

Lighthill's case was not that the programs failed. It was that they succeeded on small problems and that the search space grows exponentially, so success on a toy problem tells you nothing about the real one. A technique that handles a three-block world does not scale to a warehouse. It gets exponentially worse.

He was *right about search*. And he was wrong about the field, because the way out was not a better search algorithm — it was a different paradigm entirely, and it arrived thirty-nine years later.

British AI funding was cut across universities. Researchers left the field or left the country. In the United States, DARPA reduced its support in the same period. The reports gave the sceptics the ammunition; the promises had loaded the gun.

The second winter, 1987-1993



What I am not saying

I hold no position on whether the current wave breaks. Anyone who tells you confidently either way is guessing. What I do hold firmly: the capabilities we will build in this course work *today*, on a laptop, and your ability to build them does not depend on the forecast. That is why this course teaches you to construct things rather than to have opinions about them.



The same cause, twice

Neither collapse was caused by a technical result. Both were caused by the distance between what had been promised and what had been delivered, and in both cases the promise was made by people who genuinely believed it. Hold on to that when you read a press release this year. The question to ask is not "is this impressive?" but "what exactly was promised, by when, and who is holding the receipt?"

PART 3 • 8 MINUTES

The uncomfortable fact: the mathematics was already there

3

The part of this history that surprises people who have only read about 2012.

There is a popular story in which deep learning was invented around 2012 by a small number of people who had a brilliant new idea. Look again at the timeline and count the years.

1943

1958

1986

1998

Backpropagation — the algorithm you will derive by hand in session 14, the algorithm that trains every model in this course and every frontier model in existence — was published in 1986. **Twenty-six years before AlexNet.**

Convolutional networks were not a promising laboratory idea in 2012. They were a shipped product. LeCun's networks were reading handwritten zip codes at Bell Labs in 1989, were deployed commercially in cheque-reading machines from 1993, and by the late 1990s were reading more than one cheque in ten in the United States — around twenty million cheques a day by 2001. Filters, pooling, a learned hierarchy, trained by backpropagation. Every structural idea in AlexNet was present in LeNet-5.

SO ASK THE OBVIOUS QUESTION

If the neuron was 1943, the training algorithm was 1986, and working convolutional networks were reading a tenth of America's cheques by 1998 — **why did nothing happen until 2012?**

The honest answer is not a story about genius. It is a story about arithmetic, and you are going to do the arithmetic yourself in the next fifteen minutes.

PART 4 • 14 MINUTES • DO THIS IN CLASS

Why 2012 and not 1989 — compute it yourself

4

Four multiplications and one division. No calculus, no code. Just the size of the job and the speed of the machine.

Here are the published facts about training AlexNet in 2012. Nothing here is a secret; it is all in the paper.

Quantity	Value	Where it comes from
Operations in one forward pass	$\approx 1.1 \times 10^9$	The architecture: 5 convolutional + 3 fully-connected layers
Cost of forward + backward	$\approx 3 \times \text{forward}$	Standard rule of thumb; you will see why in session 14
Training images	1.2×10^6	The ImageNet training set, 1000 classes
Passes over the data (epochs)	90	Reported in the paper

STEP 1 — HOW BIG IS THE JOB?

arithmetic — do it on paper, then check in a notebook

PYTHON

```
work_per_image = 3 * 1.1e9          # forward + backward
total          = work_per_image * 1.2e6 * 90

print(total)      # 3.564e+17 → about 360 quadrillion operations
```

STEP 2 — HOW FAST WAS THE MACHINE?

AlexNet was trained on two NVIDIA GTX 580 graphics cards, each with 3 GB of memory. Peak throughput of one card is about 1.58×10^{12} operations per second. Real code never reaches peak; assume 30%, which is generous but not absurd.

```
speed_2012 = 2 * 1.581e12 * 0.30    # ≈ 9.5e11 ops/second
seconds     = total / speed_2012

print(seconds / 86400)    # 4.3 days
```

**Sanity check — and this is the point of doing it**

The paper reports that training took five to six days. You just estimated 4.3 days from four published numbers and one assumption. Landing that close means the model of "size of job ÷ speed of machine" is capturing something real. Now we can trust it for the counterfactual.

STEP 3 — THE COUNTERFACTUAL

Now run the same job on the hardware that was available when backpropagation was published and LeCun's networks were being built. A good scientific workstation of that era ran on the order of 10 million operations per second.

Before you compute it — write down your guess

How long would that same training run have taken in 1989? Commit to a number. Most people are wrong by a factor of a hundred.

$3.564e17 / 1e7 = 3.56e10$ seconds

Divide by the number of seconds in a year (about 3.16×10^7):

≈ **1,130 years**.

Start the job in 1989 and it finishes in the year 3119. Even on a machine ten times faster than the one assumed, it is still 113 years. There is no amount of cleverness, funding or persistence that closes a gap of that size.

This is the answer to "why 2012 and not 1989". Not a missing idea. Not a lack of talent. The job was simply, arithmetically, impossible to run — and the people trying were not failing, they were early.

4.3 days

ON 2012 HARDWARE · ESTIMATED

5–6 days

ON 2012 HARDWARE · REPORTED

≈1 130 years

THE SAME JOB IN 1989

**And notice where the hardware came from**

Nobody built the GTX 580 for artificial intelligence. It was built to render video games. It happens to do the one operation a neural network layer needs — the same multiply-and-add, over and over, on thousands of numbers at once. The field's decisive instrument was a side effect of the games industry. In session 7 you will write that operation as a single line of NumPy and see exactly why a graphics card is the right shape for it.

THE SECOND GIFT: THE DATA

Compute was half of it. The other half was ImageNet — 1.2 million labelled training images across a thousand categories, assembled over years by people paying human beings to label pictures. Backpropagation with no data teaches a network nothing. In 1989 the data did not exist either, and building it was a logistical project, not a research one.

THE HINGE OF THIS SESSION

Everything up to here is history. Everything after it is the present — and it is the same story. What made 2012 possible was not a cleverer idea but more data and more compute, and what determines who benefits in 2026 is still, largely, who has data and who has compute. Keep that in your head for the next forty minutes.

PART 5 • 8 MINUTES

What actually works, in September 2026

5

Not what is promised. Not what is demonstrated on stage. What you can rely on next week.

I am going to avoid naming a best model, and you should understand why rather than assume I am dodging. Any ranking I give you today will be wrong by December — these systems are replaced on a timescale of months. What does not go stale is the shape of the capability. So: three tiers.



RELIABLE

Transcribing speech in a well-resourced language. Translating between well-resourced languages. Classifying images into known categories. Drafting, summarising and rewriting text. Writing routine code and explaining unfamiliar code. Extracting structured fields from documents.



WORKS WITH SUPERVISION

Multi-step reasoning. Anything numerical. Any claim of fact, citation or reference. Long agentic tasks with several tools. Code touching an unfamiliar codebase. All of this is useful and none of it is trustworthy unattended.



DOES NOT WORK

Knowing what it does not know. Reliable arithmetic without a tool. Genuine novelty outside the training distribution. Any language with little text on the internet. Guarantees of any kind.

THE MOST USEFUL SINGLE SENTENCE ABOUT THESE SYSTEMS

They are excellent at tasks where *plausible* and *correct* coincide, and dangerous exactly where those two come apart.

Writing a paragraph in the style of a report: plausible is correct. Citing the paper that paragraph refers to: plausible is *not* correct, and the system cannot tell the difference — because nothing in its training taught it to represent that difference. Session 27 explains why in mechanical terms. For now, use the sentence as a filter.

How to read a leaderboard

This is a skill, and it will still be worth something in ten years. A ranking is worth about six weeks.

You will be shown leaderboards constantly — in vendor pitches, in the press, in papers, by colleagues arguing about which model to adopt. Four things go wrong with them, and once you can name all four you will read them better than most people who cite them.

Problem 1 — Saturation

When every frontier model scores above 88% on a benchmark, the remaining differences are inside the noise. MMLU, for years the most-cited general capability test, is functionally saturated: the gap between first and fourth place at the top of that board tells you nothing you can act on. GSM8K, the grade-school maths benchmark, is now answered at around 99% by frontier systems, which makes it useful only for evaluating small models — it can no longer distinguish large ones.

A saturated benchmark does not stop being published. It stops being informative, which is worse, because the number still looks like information.

Problem 2 — Contamination

A benchmark is a fixed set of questions published on the internet. Models are trained on the internet. You can see the difficulty.

If the test items are in the training data, the score measures recall rather than reasoning — and it collapses the moment you hand the model a fresh question of the same type. This is not hypothetical or rare. Training corpora are now so large that the labs themselves have limited certainty about what is inside them.



A concrete example worth remembering

In February 2026, OpenAI's evaluations team announced it would stop reporting SWE-bench Verified — for three years the unofficial scoreboard for whether a model can write real software. Not because their score was bad. Because contamination had made the number stop meaning anything. When the organisation with the strongest incentive to cite a favourable benchmark walks away from it, that is worth more than any single score on it.

Problems 3 and 4 — who built it, and the column nobody reads

Ask who assembled the board. A vendor's own comparison selects the benchmarks on which that vendor wins, and does so without lying about a single number. An independent board is better, but someone still chose which tasks count as "capability", and that choice is an argument rather than a measurement.

Then look for the column that is almost never in the headline: **cost, latency, and whether the thing can run where you need it to run**. A model two points better and forty times more expensive is the worse choice for nearly every real deployment, and a model you cannot run offline is no help at all in a field where the network is unreliable. The ranking answers a question you were not asking.

Do this now — five minutes

A vendor sends you this table to justify adopting Model A. Find as many problems as you can before opening the answer.

Model	MMLU	GSM8K	Arena rank	Cost / 1M tokens
Model A (<i>ours</i>)	91.2	98.7	#1	\$18.00
Model B	90.4	98.1	#3	\$2.50
Model C	89.8	99.0	#2	\$0.80

How many did you find?

There are at least six. Write your list first.

- 1. Both benchmarks are saturated.** All three models are above 88% on MMLU and above 98% on GSM8K. The 0.8-point MMLU spread between A and B is not a capability difference you can act on; it is noise dressed as a decimal.

2. Both benchmarks are public and static. Contamination is not merely possible, it should be your default assumption. These numbers partly measure memory.

3. Model C beats Model A on GSM8K — and the table is arranged so you read left to right and stop. Always read every column, including the ones that do not flatter the pitch.

4. The arena rank has no confidence interval. Rank #1 and rank #3 in a preference arena routinely sit inside overlapping intervals, which means the ordering may not be statistically real at all. Ask for the interval; if it is not offered, the rank is decoration.

5. Model A costs 22.5× Model C. For a 1.4-point MMLU difference that is inside the noise. On any workload with volume, this is the only column that matters.

6. The vendor made the table. No harness is stated, no evaluation date, no methodology, and the benchmarks chosen are the ones their model wins. Note also what is *absent*: nothing here tests your actual task.

The professional answer is not "Model C". It is: *none of this tells me anything. Give me two hundred examples of my own task and I will evaluate all three myself.* That is what session 21 teaches you to do properly.

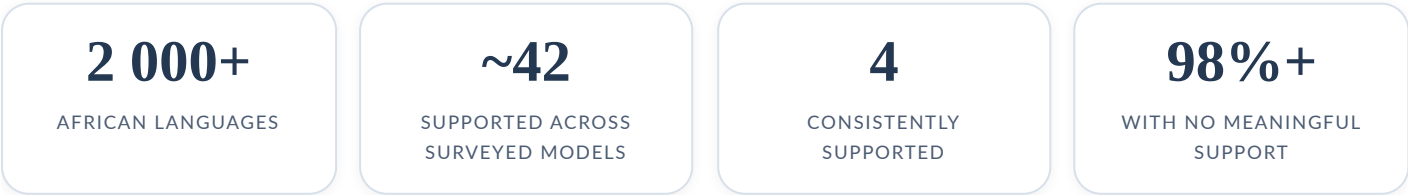
PART 7 • 8 MINUTES

The gap that is on no leaderboard

Everything above was about how well these systems work. Now: for whom.

7

Africa has more than two thousand languages. A 2025 survey of six large language models, eight smaller multilingual models and six African-specialised models found roughly forty-two African languages supported in total — and only four appearing consistently across models: Amharic, Swahili, Afrikaans and Malagasy. That leaves **over 98% of African languages with essentially no support**. Twenty writing systems in active use are not handled at all.



And where support exists, it is thinner than the marketing suggests. On IrokoBench, a benchmark built specifically for African languages, the best model tested showed a performance gap of up to **28 points against English** and up to **19 points against French**.

Not a rounding difference — a different product.



This is not an abstract concern in this room

In one large study, Bambara — a Mande language, closely related to the Dioula spoken widely across this country — was among the languages whose model performance correlated *least* with every other language tested. The researchers attributed this to Mande grammar differing sharply from the language families that dominate training data. Techniques that transfer well between European languages do not transfer here. The problem is not that nobody got around to Dioula. It is structural.

Why this happens, and why it is the same story as session 3

On Tuesday we established that the 2012 breakthrough waited twenty-six years for compute and labelled data, and that both arrived from outside the field. The language gap is the identical mechanism, running now.

Training a frontier model needs trillions of tokens of text and a compute budget in the tens to hundreds of millions of dollars. A language with little text on the internet cannot supply the first, and most institutions cannot supply the second. Even tokenisation works against you: the tokenizers are fitted to high-resource languages, so a low-resource language is chopped into more pieces, costs more per sentence, and fits worse in a context window. The disadvantage compounds at every layer.

THE CONSEQUENCE, STATED WITHOUT COMFORT

If the binding constraint on this field is infrastructure rather than insight, then whoever builds the infrastructure decides what gets built. Nobody is going to assemble a Dioula corpus on your behalf.

That is not a complaint. It is a description of an opening — the kind that, in 2009, was called "a labelled image dataset nobody wants to build."

CLOSING · 6 MINUTES

Côte d'Ivoire, and why this course exists

The national position, the ambition, and the honest distance between them.

8

In the 2023 Oxford Insights Government AI Readiness Index, Côte d'Ivoire ranked **138th of 193 countries**. That assessment is part of what produced the response.

In March 2025 the Ministry of Digital Transition and Digitalisation, with support from the World Bank, formalised the **Stratégie Nationale de l'Intelligence Artificielle — SNIA 2030**. Its stated aim is to position Côte d'Ivoire as a regional leader in AI adoption in West Africa by 2030.

SNIA 2030	
Three pillars	Investment · Inclusion · Governance
Budget	Over 1,000 billion FCFA — roughly USD 1.8 billion — to 2030
Institutions	A National AI Agency (ANIA), and a national committee for AI and data governance
Priority sectors	Agriculture, health, education, public administration

Governance instrument	A "Safe AI Côte d'Ivoire" certification, developed with UNESCO
Education aim	AI fundamentals in school curricula; new talent trained through international partnerships
Companion strategy	SNGD 2030, on national data governance — presented alongside it



Read the pillars against what you learned today

Investment answers the compute constraint. Inclusion answers the language and data constraint. Governance answers the deployment constraint that killed MYCIN — a system that worked and was never used, because nobody had resolved who is responsible when it is wrong. The three pillars are not slogans; each one names a real bottleneck you have now met by its technical name.

Now the honest part. A strategy is a document. 138th place is a measurement. The distance between them is closed by people who can build things, and that is a smaller group than the country needs. There are twenty-six sessions left in this course, and at the end of them you will have written a neural network from nothing, trained it, and built a small language model. That is not a complete answer to a national strategy. It is, however, exactly the kind of person the strategy is short of.

WHY THIS COURSE IS TAUGHT THE WAY IT IS

You could learn to call an API in an afternoon, and thousands of people here already have. That skill is real and it is also common, and it leaves you dependent on infrastructure built elsewhere for problems defined elsewhere.

Understanding what is inside the box is what lets you build one for a problem nobody else has bothered with. That is why sessions 9 to 14 have no libraries in them.

Before Friday

1. **Finish the compute calculation** from part 4 if you did not get to the end of it in class, and write the ratio down as a single number. You will be asked for it.
2. **Run one prompt on two different frontier models** — the same prompt, word for word — and bring both answers. Something with a right answer is best: a factual question about Côte d'Ivoire, or a small piece of arithmetic. Where they diverge is the interesting part, and we will use several of these on Friday.
3. **Nothing to install.** Friday is the first coding session and the environment is already set up from session 1. If your setup did not complete, come and see me before Friday rather than during it.



Session 4 — Friday 18 September, 11:30

Python I: values, functions, and the loop. The first of six sessions on the tools. We start from the beginning — variables, a function that takes arguments and returns something, if/else, and the loop with an accumulator — and we stop there. If you have programmed before, the session will feel slow for twenty minutes and then it will not. If you have not, this is the session that decides the rest of your term, and it is built for you.

Sources for the figures quoted today

- **AlexNet** — 15.3% vs 26.2% top-5 error, 60M parameters, two GTX 580 cards, 90 epochs over 1.2M images, 5–6 days training: Krizhevsky, Sutskever & Hinton, "ImageNet Classification with Deep Convolutional Neural Networks", NeurIPS 2012.
- **The Lighthill report** — combinatorial explosion and the UK funding cuts: J. Lighthill, "Artificial Intelligence: A General Survey", British Science Research Council, 1973. A transcript of the televised debate that followed is at github.com/Dicklesworthstone.

- **The ALPAC report** — machine translation slower, less accurate and more expensive than human translation: Automatic Language Processing Advisory Committee, National Research Council, 1966.
- **Backpropagation** — Rumelhart, Hinton & Williams, "Learning representations by back-propagating errors", *Nature* 323, 1986.
- **LeNet and the cheque readers** — commercial deployment from 1993, over 10% of US cheques by the late 1990s: LeCun, Bottou, Bengio & Haffner, "Gradient-Based Learning Applied to Document Recognition", *Proc. IEEE*, 1998; see also [Wikipedia](#), "[LeNet](#)".
- **XCON** — 2,500 rules, ~\$25M annual saving; growth to ~17,000 rules and 31,000 components by 1989: [Wikipedia](#), "[Xcon](#)"; [arXiv:cs/0312020](#).
- **GTX 580 throughput** — approximately 1.58 TFLOPS single precision, manufacturer specification. The 30% efficiency figure is our assumption, not a published number.